

Telecom Insights Generator

Production Grade Low Level Design (LLD)

Problem statement: How can telecom executives and analysts transform raw usage data into actionable insights using GenAI, without relying on manual data processing or deep technical expertise?

Primary stack

Spring Boot, Spring AI, PostgreSQL, pgvector, Redis, Kafka, REST APIs

AI strategy

Spring AI as the orchestrator for agent routing, prompt management, retrieval, and response synthesis

Target users

Telecom executives, analysts, operations teams, and support engineers

Version 1.0

This document is written as an implementation-ready LLD so a new team member can build and extend the system with minimal context.

1. Document Overview

This LLD translates the HLD concept into buildable Spring Boot components. The design follows a layered enterprise pattern: ingestion, enrichment, AI orchestration, insight generation, and presentation. The AI layer is intentionally split into specialized agents to keep prompts focused and the code maintainable.

1.1 Scope

The first release focuses on natural-language analytics over telecom usage data, schema-aware NLQ to SQL generation, narrative insight generation, anomaly detection on KPI streams, and forecast retrieval from pre-trained models. The system will support both interactive user questions and automated background analysis.

1.2 Non-scope for v1

The v1 implementation does not train large language models, does not replace the source systems, and does not expose raw databases directly to the UI. The system consumes curated data and schema metadata from upstream services.

1.3 Design principles

- Keep each AI responsibility isolated in its own agent class.
- Use Spring AI for orchestration, prompt templates, tool calling, and retrieval integration.
- Keep NLQ SQL generation guarded by schema metadata and validation rules.
- Separate operational data from vectorized metadata and embeddings.
- Return answer payloads with traceable evidence and confidence metadata.

2. Architecture Summary

The reference architecture maps to five layers. Raw telecom data flows from source systems into ingestion services, is transformed and stored in analytic databases, is indexed into vector storage for retrieval, is processed by Spring AI orchestrated agents, and is surfaced through a React front end.

Layer	Responsibilities	Key components
1. Source systems	Network KPIs, usage records, CRM signals, billing data, site performance feeds	External APIs, files, streaming topics
2. Ingestion and processing	Validate, normalize, enrich, store, publish events	Ingestion job, transformer, validator, Kafka producer
3. Data and vector stores	Persist facts and searchable metadata	PostgreSQL, pgvector, Redis cache
4. AI orchestration	Route queries and coordinate agents	Spring AI orchestrator, prompt templates, tool functions
5. Presentation	Expose conversational and analytical insights	REST API, React UI, optional export service

2.1 User request lifecycle

A user enters a natural-language query, for example: 'Why is churn low in Andhra Pradesh this month?' The controller hands the request to the Spring AI orchestrator. The orchestrator classifies intent, picks the right agent path, retrieves relevant schema or historical context, performs guarded SQL or retrieval, synthesizes the response, and returns an answer with evidence and trace metadata.

```
User -> Query API -> Spring AI Orchestrator
      -> Intent classification
      -> Agent routing
      -> NLQ agent / Insight agent / Forecast agent / Anomaly agent
      -> Retrieval / SQL / model call
      -> Response synthesis
      -> UI
```

3. Functional Components

3.1 Query API and orchestration

The Query API is the single entry point for interactive analysis. It accepts the user question, optional filters, and user context. Spring AI is used as the orchestrator to determine whether the request should be answered using NLQ, historical retrieval, anomaly detection, or forecasting.

3.2 NLQ Agent

The NLQ agent converts natural language to a safe SQL query using schema-aware retrieval. It should never write unrestricted SQL. It uses vector search over schema metadata, table descriptions, and sample business questions, then asks the LLM to generate only a constrained SELECT query.

3.3 Insight Agent

The Insight agent takes structured query results and turns them into business language. It computes trends, comparisons, deviations, and root-cause hypotheses. This agent is responsible for executive-friendly narration, not SQL generation.

3.4 Anomaly Agent

The Anomaly agent reads streaming or near-real-time KPI data and detects unexpected drops or spikes. The implementation can start with a simple Isolation Forest service or an external ML endpoint. It returns anomaly score, severity, impacted region, and recommended next action.

3.5 Forecast Agent

The Forecast agent consumes trained model output to predict churn, demand, or traffic growth. The model can live outside the Spring Boot service, but the orchestration and response shaping remain in Spring AI service code.

Agent	Input	Output	Typical trigger
NLQ	Human question + filters	Validated SQL + query result	What is / why is / show me
Insight	Query result + business context	Narrative explanation	Summarize / explain / compare
Anomaly	Streaming KPI windows	Anomaly score + alert	Unexpected KPI movement
Forecast	Historical features	Forecast curve / values	Predict / forecast / estimate

4. Technology Stack and Dependencies

The implementation uses Spring Boot as the service runtime and Spring AI as the orchestrator abstraction. PostgreSQL stores curated data and metadata. pgvector stores embeddings for schema and business-context retrieval. Redis supports caching for repeated queries and short-lived conversation state. Kafka or another stream bus can be used for anomaly feeds.

4.1 Maven dependencies

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-openai</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
  </dependency>

  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
  </dependency>
</dependencies>
```

4.2 application.yml sketch

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/tig
    username: tig_user
    password: tig_pwd
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      chat:
        options:
          model: gpt-4.1-mini
    vectorstore:
      pgvector:
        initialize-schema: true
  data:
    redis:
      host: localhost
      port: 6379

tig:
  llm:
    nlq:
      temperature: 0.0
      max-tokens: 700
    insight:
      temperature: 0.4
  safety:
    allow-only-select: true
```

5. Recommended Spring Boot Package Structure

The package structure keeps orchestration, agents, data access, and configuration separate so the code can scale cleanly as additional AI skills are added.

```
com.tig
├── TigApplication.java
├── controller
│   └── QueryController.java
├── orchestrator
│   ├── AgentOrchestrator.java
│   └── AgentRouter.java
├── agents
│   ├── nlq
│   │   ├── NlqAgent.java
│   │   ├── NlqAgentImpl.java
│   │   ├── SqlGenerationService.java
│   │   └── SqlValidator.java
│   ├── insight
│   │   ├── InsightAgent.java
│   │   └── InsightAgentImpl.java
│   ├── anomaly
│   │   ├── AnomalyAgent.java
│   │   └── AnomalyAgentImpl.java
│   └── forecast
│       ├── ForecastAgent.java
│       └── ForecastAgentImpl.java
├── service
│   ├── LlmService.java
│   ├── EmbeddingService.java
│   ├── SchemaRetrievalService.java
│   └── QueryCacheService.java
├── repository
│   ├── UsageMetricsRepository.java
│   ├── SchemaMetadataRepository.java
│   └── ConversationTraceRepository.java
├── model
│   ├── QueryRequest.java
│   ├── QueryResponse.java
│   ├── QueryIntent.java
│   ├── QueryResult.java
│   ├── AnswerEvidence.java
│   └── AgentType.java
├── config
│   ├── AiConfig.java
│   ├── VectorStoreConfig.java
│   ├── RedisConfig.java
│   └── SecurityConfig.java
└── util
    ├── PromptTemplates.java
    └── QuerySanitizer.java
```

5.1 Class responsibility summary

Class	Role
QueryController	HTTP entry point for ask/query operations
AgentOrchestrator	Routes each query to the correct agent path
NlqAgent	Builds safe SQL from the user query
InsightAgent	Generates executive narrative from result sets
AnomalyAgent	Produces anomaly detection outputs
ForecastAgent	Calls forecasting model and formats results
SchemaRetrievalService	Fetches relevant tables and fields from vector store

6. API Contracts

6.1 Query API

POST /api/v1/insights/query

Request:

```
{
  "question": "Why is churn low in Andhra Pradesh this month?",
  "region": "Andhra Pradesh",
  "timeWindow": "CURRENT_MONTH",
  "userRole": "EXECUTIVE"
}
```

Response:

```
{
  "queryId": "b2a3f7c1",
  "answer": "Churn is lower because...",
  "agentPath": ["INTENT", "NLQ", "INSIGHT"],
  "evidence": [
    {
      "source": "usage_metrics",
      "description": "AP tower uptime improved by 3.1%"
    }
  ],
  "confidence": 0.87
}
```

6.2 Anomaly API

POST /api/v1/insights/anomalies/stream

POST /api/v1/insights/anomalies/check

Request:

```
{
  "metric": "call_drop_rate",
  "region": "Andhra Pradesh"
}
```

Response:

```
{
  "status": "ANOMALY_DETECTED",
  "severity": "HIGH",
  "score": 0.94
}
```

6.3 Forecast API

POST /api/v1/insights/forecast

Request:

```
{
  "metric": "churn_rate",
  "region": "Andhra Pradesh",
  "horizonDays": 30
}
```

Response:

```
{
  "forecastValues": [2.1, 2.0, 1.9],
  "trend": "DOWNWARD",
  "model": "Prophet"
}
```

7. Core Runtime Flow

The following is the operational runtime sequence for a natural-language analytics query. This is the central flow the team should implement first.

Step	What happens	Implementation note
1	Query reaches QueryController	Validate required fields and assign correlation ID
2	Spring AI orchestrator extracts intent	Use a structured output prompt to classify query type
3	Router selects the right agent path	NLQ, Insight, Forecast, or Anomaly, or a chain
4	Relevant context is retrieved	Use pgvector for schema and business context retrieval
5	Agent executes safe tool or DB call	SQL is validated before execution
6	Answer is synthesized	Return evidence, confidence, and traces
7	Response is cached and logged	Store the trace for observability and reuse

7.1 Sequence narrative

For a question such as 'What explains lower churn in Andhra Pradesh?', the orchestrator first classifies the request as an analytical NLQ request. It retrieves the most relevant schema descriptions, identifies the churn fact table and supporting dimensions, generates a constrained SQL statement, validates it, executes it, and then passes the result set into the Insight agent for executive-grade narration.

8. Production-Ready Code Skeleton

8.1 Query controller

```
@RestController
@RequestMapping("/api/v1/insights")
@RequiredArgsConstructor
public class QueryController {

    private final AgentOrchestrator orchestrator;

    @PostMapping("/query")
    public ResponseEntity<QueryResponse> query(@Valid @RequestBody QueryRequest request) {
        QueryResponse response = orchestrator.handle(request);
        return ResponseEntity.ok(response);
    }
}
```

8.2 Orchestrator with Spring AI

```
@Service
@RequiredArgsConstructor
public class AgentOrchestrator {

    private final QueryRouter queryRouter;
    private final QueryCacheService cacheService;
    private final ConversationTraceService traceService;

    public QueryResponse handle(QueryRequest request) {
        String cacheKey = cacheService.buildKey(request);

        Optional<QueryResponse> cached = cacheService.get(cacheKey);
        if (cached.isPresent()) {
            return cached.get();
        }

        QueryIntent intent = queryRouter.classify(request);
        AgentExecutionPlan plan = queryRouter.plan(intent);

        QueryResponse response = queryRouter.execute(plan, request, intent);

        traceService.save(request, intent, plan, response);
        cacheService.put(cacheKey, response);

        return response;
    }
}
```

8.3 Intent model

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class QueryIntent {
    private String topic;
    private String region;
    private String timewindow;
    private AgentType agentType;
    private boolean needsRag;
    private boolean needsForecast;
    private boolean needsAnomalyCheck;
}
```

8.4 NLQ agent with guarded SQL generation

```
@Service
@RequiredArgsConstructor
public class NlqAgentImpl implements NlqAgent {

    private final SchemaRetrievalService schemaRetrievalService;
    private final LlmService llmService;
    private final SqlValidator sqlValidator;
    private final JdbcTemplate jdbcTemplate;

    @Override
    public QueryResult fetchData(QueryIntent intent) {
        List<SchemaChunk> chunks = schemaRetrievalService.findRelevantChunks(intent);

        String sql = llmService.generateSql(intent, chunks);
        sqlValidator.validateSelectOnly(sql);

        List<Map<String, Object>> rows = jdbcTemplate.queryForList(sql);

        return QueryResult.builder()
            .rows(rows)
            .sql(sql)
            .build();
    }
}
```

8.5 SQL validator

```
@Component
public class SqlValidator {

    public void validateSelectOnly(String sql) {
        String normalized = sql.trim().toLowerCase(Locale.ROOT);

        if (!normalized.startsWith("select")) {
            throw new IllegalArgumentException("Only SELECT queries are allowed");
        }

        List<String> forbidden = List.of("insert ", "update ", "delete ", "drop ", "alter ", "truncate ");
        for (String word : forbidden) {
            if (normalized.contains(word)) {
                throw new IllegalArgumentException("Unsafe SQL detected");
            }
        }
    }
}
```

8.6 Insight agent

```
@Service
@RequiredArgsConstructor
public class InsightAgentImpl implements InsightAgent {

    private final LlmService llmService;

    @Override
    public String generateInsights(QueryResult result, QueryIntent intent) {
        return llmService.summarizeResult(
            intent,
            result.getRows(),
            result.getSql()
        );
    }
}
```

9. Data Model and RAG Design

The system uses PostgreSQL for relational analytics and pgvector for semantic retrieval over schema metadata and operational context. The vector store should not be used as the source of truth for raw transactional facts. It is a discovery layer that helps the orchestrator find the right facts quickly.

9.1 Suggested database tables

Table	Purpose
usage_metrics	Curated telecom usage facts, by region, time, product, and metric
schema_metadata	Table descriptions, column definitions, synonyms, and business terms
conversation_trace	Query traces, chosen agents, confidence, and latency
insight_cache	Frequently requested answers for low-latency responses
anomaly_events	Detected stream anomalies and severity scores

9.2 pgvector schema example

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE schema_metadata (
  id BIGSERIAL PRIMARY KEY,
  table_name VARCHAR(128) NOT NULL,
  column_name VARCHAR(128),
  business_description TEXT NOT NULL,
  embedding VECTOR(1536),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_schema_metadata_embedding
ON schema_metadata USING ivfflat (embedding vector_cosine_ops);
```

9.3 Retrieval strategy

The retrieval layer should first use the user intent to locate relevant business terms, then use similarity search over schema metadata, then optionally apply filters for region, time window, and line of business. This keeps the LLM from inventing table names or joining incorrect relations.

```
Question -> embedding
          -> schema_metadata similarity search
          -> top-k relevant tables and columns
          -> prompt context
          -> SQL generation
```

10. Prompting and Safety Controls

Prompt engineering is part of the code design. Each agent should have a dedicated prompt template and a dedicated output schema. The NLQ prompt must be strict, concise, and focused on safe SQL generation. The Insight prompt should prefer evidence-backed language over speculation.

10.1 NLQ prompt template

You are a telecom SQL assistant.
Use only the provided schema context.
Generate exactly one PostgreSQL SELECT query.
Rules:
- Do not use INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE.
- Do not reference tables not present in schema context.
- Prefer explicit joins and readable aliases.
- Return SQL only.

User question: {question}
Schema context: {schemaContext}

10.2 Insight prompt template

You are a telecom executive analyst.
Explain the result in business language.
Use only the supplied data and evidence.
Highlight trends, comparisons, and likely reasons.
If evidence is weak, say so clearly.
Question: {question}
Result rows: {rows}
Evidence: {evidence}

10.3 Response contract

```
@Data
@Builder
public class QueryResponse {
    private String queryId;
    private String answer;
    private List<AnswerEvidence> evidence;
    private List<String> agentPath;
    private Double confidence;
    private Long latencyMs;
}
```

10.4 SQL safety strategy

- Generate SELECT-only SQL.
- Validate table names against schema metadata.
- Reject queries with write or DDL keywords.
- Apply row limits for large result sets.
- Log generated SQL for audit and tuning.

11. Non-Functional Requirements

- **Latency:** return standard analytical responses within a few seconds, with caching for repeated queries.
- **Reliability:** degrade gracefully if the LLM is unavailable by returning a controlled error or cached response.
- **Security:** use authentication, authorization, query validation, and secret management.
- **Auditability:** store prompt versions, SQL output, confidence, and response traces.
- **Scalability:** separate sync query service from async anomaly processing.

11.1 Observability

```
traceId -> requestId
llm.model
llm.tokens.input
llm.tokens.output
retrieval.topK
sql.latencyMs
db.latencyMs
response.confidence
```

11.2 Error handling

```
if (intent == null) {
    return fallback("Could not understand the request.");
}
if (schemaContext.isEmpty()) {
    return fallback("No relevant schema found for the query.");
}
try {
    sqlValidator.validateSelectOnly(sql);
} catch (Exception e) {
    return fallback("Generated query was unsafe or invalid.");
}
```

11.3 Testing plan

Test type	Focus
Unit tests	Prompt builders, SQL validator, router, cache
Integration tests	Spring Boot + PostgreSQL + pgvector + Redis
Contract tests	API request and response payloads
LLM regression tests	Prompt version changes and answer consistency

12. Implementation Roadmap for the Team

A new team member should build the project in the following order. This order keeps the application stable and reduces rework.

1. Create the base Spring Boot project with the package structure shown above.
2. Wire PostgreSQL, Redis, and Spring AI configuration in application.yml.
3. Implement the QueryController and QueryRequest/QueryResponse models.
4. Implement schema metadata ingestion and pgvector indexing.
5. Build the NLQ agent with retrieval, SQL generation, and SQL validation.
6. Build the Insight agent for narration and evidence formatting.
7. Add the Forecast and Anomaly agents, even if initially backed by stub services.
8. Persist query traces, cache repeated responses, and add metrics.
9. Add tests and establish prompt versioning before production rollout.

12.1 Minimal first milestone

The very first milestone should support one end-to-end path: natural language question -> intent classification -> schema retrieval -> SQL generation -> database execution -> answer synthesis. Once this works reliably, the remaining agents can be layered in.

12.2 Example day-1 implementation order

Day 1: Spring Boot skeleton, config, controller, DTOs
Day 2: schema_metadata table, vector indexing, retrieval service
Day 3: NLQ prompt + SQL validator + DB execution
Day 4: Insight agent + response contract + trace persistence
Day 5: cache + tests + error handling

End of document.